

silver_to_gold

November 17, 2025

1 ETL da camada silver para camada gold

Esta célula importa todas as bibliotecas necessárias para o processo de ETL. Aqui são carregados os pacotes para manipulação de dados (pandas), conexão com o banco de dados PostgreSQL (psycopg), controle de mensagens de erro (sys) e tratamento de avisos (warnings). Ela deve ser executada antes de qualquer outra célula, pois fornece as dependências básicas que serão usadas nas etapas de Extract, Transform e Load.

```
[1]: import pandas as pd
import numpy as np
import psycopg
from psycopg import connect, sql
import sys
import warnings
import math
from dotenv import load_dotenv
import os

warnings.filterwarnings('ignore')
```

2 1. Extract

Esta célula define as configurações de conexão com o banco de dados PostgreSQL e monta a consulta SQL que será usada para extrair os dados. Ela cria variáveis com credenciais, monta o nome completo da tabela (schema.tabela) e gera a query SELECT * FROM silver.listings, além de preparar a connection string usada na etapa de conexão.

```
[2]: DB_SCHEMA = "silver"
TABLE_NAME = "listings"

TABLE_FULL_NAME = sql.SQL("{}.{}`).format(
    sql.Identifier(DB_SCHEMA),
    sql.Identifier(TABLE_NAME)
)

def get_db_connection_info():
    load_dotenv()
    url = os.getenv('DB_URL')
```

```

db_env = os.getenv('DB_ENV')
if url is not None and db_env == 'prod':
    return url

# credenciais do banco de dados local
DB_USER = "postgres"
DB_PASSWORD = "postgres"
DB_HOST = "localhost"
DB_PORT = "5433"
DB_NAME = "airbnb"

return f"host={DB_HOST} dbname={DB_NAME} user={DB_USER}␣
↳password={DB_PASSWORD} port={DB_PORT}"

query_object = sql.SQL("SELECT * FROM {}").format(TABLE_FULL_NAME)

connection_string = get_db_connection_info()

```

Esta célula executa a extração dos dados do banco PostgreSQL. Ela estabelece a conexão usando as configurações definidas anteriormente, converte o objeto SQL em uma query legível, executa a consulta e carrega o resultado no DataFrame df. Em caso de falha na conexão ou na leitura, exibe uma mensagem de erro detalhada e encerra o processo.

```

[3]: try:
    print("Estabelecendo conexão...")
    with connect(connection_string) as conn:
        print("Conexão estabelecida.")
        query_string = query_object.as_string(conn)
        print(f"Executando query: {query_string}")
        df = pd.read_sql_query(query_string, conn)
        print("\nDados carregados do banco para o DataFrame com sucesso!")
    except psycopg.Error as e:
        print(f"\n--- Ocorreu um erro ao conectar ou ler o banco de dados ---")
        print(f"Erro: {e}")
        sys.exit(1)
    except Exception as e:
        print(f"\n--- Ocorreu um erro inesperado ---")
        print(f"Erro: {e}")
        sys.exit(1)

```

Estabelecendo conexão...

Conexão estabelecida.

Executando query: SELECT * FROM "silver"."listings"

Dados carregados do banco para o DataFrame com sucesso!

Estas células exibem um resumo simples do resultado da extração, mostrando o número total de registros carregados no DataFrame df, as primeiras três tuplas e os tipos de cada dado. Elas

servem para confirmar visualmente que a consulta foi executada com sucesso e quantas linhas foram retornadas do banco.

```
[4]: print(f"Total de linhas carregadas: {len(df)}")
```

Total de linhas carregadas: 99417

```
[5]: df.head(3)
```

```
[5]:
```

	id	host_id	name \
0	1001254	80014485718	Clean & quiet apt home by the park
1	1002102	52335172823	Skylit Midtown Castle
2	1002403	78829239556	THE VILLAGE OF HARLEM...NEW YORK !

	host_identity_verified	host_name	neighbourhood_group	neighbourhood \
0	False	Madaline	Brooklyn	Kensington
1	True	Jenna	Manhattan	Midtown
2	True	Elise	Manhattan	Harlem

	lat	long	instant_bookable	... service_fee	minimum_nights \
0	40.64749	-73.97237	False	193.0	10
1	40.75362	-73.98377	False	28.0	30
2	40.80902	-73.94190	True	124.0	3

	number_of_reviews	last_review	reviews_per_month	review_rate_number \
0	9	2021-10-19	0.21	4.0
1	45	2022-05-21	0.38	4.0
2	0	None	NaN	5.0

	calculated_host_listings_count	availability_365 \
0	6	286
1	2	228
2	1	352

	house_rules	has_house_rules
0	Clean up and treat the home the way you'd like...	True
1	Pet friendly but please confirm with me if the...	True
2	I encourage you to use my kitchen, cooking and...	True

[3 rows x 24 columns]

```
[6]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 99417 entries, 0 to 99416
Data columns (total 24 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   id                                     99417 non-null  int64
```

```

1  host_id                99417 non-null  int64
2  name                   99417 non-null  object
3  host_identity_verified 99417 non-null  bool
4  host_name              99417 non-null  object
5  neighbourhood_group    99417 non-null  object
6  neighbourhood          99417 non-null  object
7  lat                   99417 non-null  float64
8  long                  99417 non-null  float64
9  instant_bookable       99417 non-null  bool
10 cancellation_policy    99364 non-null  object
11 room_type              99417 non-null  object
12 construction_year      99417 non-null  int64
13 price                  99417 non-null  float64
14 service_fee            99417 non-null  float64
15 minimum_nights         99417 non-null  int64
16 number_of_reviews      99417 non-null  int64
17 last_review            84233 non-null  object
18 reviews_per_month      84235 non-null  float64
19 review_rate_number     99417 non-null  float64
20 calculated_host_listings_count 99417 non-null  int64
21 availability_365       99417 non-null  int64
22 house_rules            99417 non-null  object
23 has_house_rules        99417 non-null  bool
dtypes: bool(3), float64(6), int64(7), object(8)
memory usage: 16.2+ MB

```

3 2. Transform

Esta célula realiza a padronização dos nomes das colunas do DataFrame para o modelo usado no Data Warehouse. Ela aplica o dicionário `mapa_colunas` para renomear os campos extraídos do banco e remove a coluna `regras_txt`, que não será utilizada nas etapas seguintes de transformação e carga.

```

[7]: mapa_colunas = {
    'id': 'id_anun',
    'host_id': 'id_anfi',
    'host_name': 'nom_anf',
    'host_identity_verified': 'ind_ver_anf',
    'calculated_host_listings_count': 'qtd_anu_anf',
    'neighbourhood_group': 'grp_bai',
    'neighbourhood': 'nom_bai',
    'lat': 'num_lat',
    'long': 'num_lon',
    'name': 'nom_anu',
    'instant_bookable': 'ind_res_ins',
    'cancellation_policy': 'des_pol_can',
    'room_type': 'tip_qto',

```

```

    'construction_year': 'ano_con',
    'minimum_nights': 'qtd_min_noi',
    'has_house_rules': 'ind_tem_reg',
    'house_rules': 'regras_txt',
    'last_review': 'dat_ava',
    'price': 'val_pre',
    'service_fee': 'val_tax_ser',
    'number_of_reviews': 'qtd_tot_ava',
    'reviews_per_month': 'qtd_ava_mes',
    'review_rate_number': 'val_not_ava',
    'availability_365': 'qtd_dia_dis'
}

df = df.rename(columns=mapa_colunas)
df = df.drop(columns=['regras_txt'], errors='ignore')

```

Esta célula define as listas de colunas que compõem cada dimensão e converte a coluna de data da última avaliação (dt_ult_rev) para o tipo datetime. Essa conversão garante que o campo temporal esteja no formato correto para gerar os atributos de tempo nas etapas seguintes do Transform.

```

[8]: col_aval = 'dat_ava'
     cols_anfi = ['nom_anf', 'ind_ver_anf', 'qtd_anu_anf']
     cols_loc = ['num_lat', 'num_lon', 'nom_bai', 'grp_bai']
     cols_prop = ['nom_anu', 'tip_qto', 'qtd_min_noi', 'des_pol_can', 'ind_res_ins',
                  'ano_con', 'ind_tem_reg']

     df[col_aval] = pd.to_datetime(df[col_aval], errors='coerce')

```

Esta célula cria o DataFrame df_aval, que representa a dimensão de tempo das últimas avaliações. Ela seleciona apenas a coluna de data, remove valores nulos e duplicados, e deriva as colunas ano, mes e trimestre a partir de dt_ult_rev, preparando os dados para carga na tabela DIM_ULTIMA_AVALIACAO.

```

[9]: df_aval = df[[col_aval]].copy()
     df_aval = df_aval.dropna(subset=[col_aval])
     df_aval = df_aval.drop_duplicates(subset=[col_aval])
     df_aval['num_ano'] = df_aval[col_aval].dt.year.astype('Int64')
     df_aval['num_mes'] = df_aval[col_aval].dt.month.astype('Int64')
     df_aval['num_tri'] = df_aval[col_aval].dt.quarter.astype('Int64')

```

Esta célula cria os DataFrames das dimensões de anfitrião, localização e propriedade. Ela seleciona as colunas correspondentes a cada dimensão, garantindo que cada conjunto contenha apenas valores únicos antes da carga no Data Warehouse.

```

[10]: df_anfi = df[cols_anfi].drop_duplicates().copy()
     df_loc = df[cols_loc].drop_duplicates().copy()
     df_prop = df[cols_prop].drop_duplicates().copy()

```

Esta célula realiza a padronização e limpeza dos campos numéricos do DataFrame. Ela converte

todas as colunas de métricas para tipo numérico, substitui valores infinitos por NaN e depois transforma todos os NaN e valores ausentes em None, garantindo que o banco de dados receba NULL corretamente durante a carga.

```
[11]: num_cols = ['qtd_ava_mes', 'qtd_tot_ava', 'val_pre', 'val_tax_ser',
    ↪ 'qtd_dia_dis', 'val_not_ava']
    for c in num_cols:
        if c in df.columns:
            df[c] = pd.to_numeric(df[c], errors='coerce')

    df = df.replace([np.inf, -np.inf], np.nan)
    df = df.where(pd.notna(df), None)
```

Esta célula exibe um resumo da etapa de transformação, mostrando quantos registros foram preparados em cada dimensão e na tabela fato base. Em seguida, utiliza df.info() para apresentar a estrutura geral do DataFrame principal, permitindo verificar tipos de dados e possíveis valores nulos antes da carga no banco.

```
[12]: print("Registros preparados para carga:")
    print(f"dim_anf: {len(df_anfi)}")
    print(f"dim_loc: {len(df_loc)}")
    print(f"dim_pro: {len(df_prop)}")
    print(f"dim_ult_ava: {len(df_aval)}")
    print(f"fat_anu (base df): {len(df)}\n\n")

    df.info()
```

Registros preparados para carga:

dim_anf: 25735

dim_loc: 65408

dim_pro: 93536

dim_ult_ava: 2436

fat_anu (base df): 99417

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 99417 entries, 0 to 99416

Data columns (total 23 columns):

#	Column	Non-Null Count	Dtype
0	id_anun	99417 non-null	int64
1	id_anfi	99417 non-null	int64
2	nom_anu	99417 non-null	object
3	ind_ver_anf	99417 non-null	bool
4	nom_anf	99417 non-null	object
5	grp_bai	99417 non-null	object
6	nom_bai	99417 non-null	object
7	num_lat	99417 non-null	float64
8	num_lon	99417 non-null	float64

```

9   ind_res_ins  99417 non-null  bool
10  des_pol_can  99364 non-null  object
11  tip_qto      99417 non-null  object
12  ano_con      99417 non-null  int64
13  val_pre      99417 non-null  float64
14  val_tax_ser  99417 non-null  float64
15  qtd_min_noi  99417 non-null  int64
16  qtd_tot_ava  99417 non-null  int64
17  dat_ava      84233 non-null  datetime64[ns]
18  qtd_ava_mes  84235 non-null  float64
19  val_not_ava  99417 non-null  float64
20  qtd_anu_anf  99417 non-null  int64
21  qtd_dia_dis  99417 non-null  int64
22  ind_tem_reg  99417 non-null  bool
dtypes: bool(3), datetime64[ns](1), float64(6), int64(7), object(6)
memory usage: 15.5+ MB

```

4 3. Load

Esta célula configura os parâmetros de conexão com o banco de dados do Data Warehouse (dw) e valida se todas as variáveis geradas na etapa de transformação estão disponíveis na memória. Ela garante que o ambiente esteja pronto antes de iniciar a fase de carga, evitando erros por falta de dados ou variáveis necessárias.

```

[13]: DB_SCHEMA_GOLD = "dw"

connection_string = get_db_connection_info()

for v in ['df', 'df_anfi', 'df_loc', 'df_prop', 'df_aval', 'cols_anfi',
         'cols_loc', 'cols_prop', 'col_aval']:
    if v not in globals():
        raise RuntimeError(f"Variável ausente: {v}")

```

Esta célula executa o script DDL responsável por criar ou recriar as tabelas do schema dw no banco de dados. Ela lê o arquivo SQL que contém a definição das tabelas e executa o comando dentro de uma conexão com o PostgreSQL, preparando a estrutura necessária para receber os dados na etapa de carga.

```

[14]: try:
        ddl_gold = open('../data_layer/gold/gold_ddl.sql').read()
    except FileNotFoundError:
        print("Erro: Arquivo 'gold_ddl.sql' não encontrado em '../data_layer/"
              "gold/gold_ddl.sql'.")
        sys.exit(1)

    with connect(connection_string) as conn:
        with conn.cursor() as cur:

```

```
cur.execute(ddl_gold)
```

Esta célula realiza a carga da dimensão Anfitrião no schema dw. Ela insere todos os registros do DataFrame df_anfi na tabela DIM_ANFITRIAO e adiciona uma linha extra com valores nulos para representar o registro “desconhecido”, armazenando sua chave substituta (unknown_anfi_key) para uso posterior na carga da tabela fato.

```
[15]: with connect(connection_string) as conn:
      with conn.cursor() as cur:
          insert_query = sql.SQL("INSERT INTO dw.dim_anf (nom_anf, ind_ver_anf,
          ↳qtd_anu_anf) VALUES (%s, %s, %s)")
          cur.executemany(insert_query, [tuple(x) for x in df_anfi.to_numpy()])
          cur.execute("INSERT INTO dw.dim_anf (nom_anf, ind_ver_anf, qtd_anu_anf)
          ↳VALUES (NULL, NULL, NULL) RETURNING srk_anf")
          unknown_anfi_key = cur.fetchone()[0]
```

Esta célula insere os dados da dimensão Localização no schema dw. Ela carrega todos os registros do DataFrame df_loc na tabela DIM_LOCALIZACAO e adiciona um registro adicional com valores nulos para representar a localização “desconhecida”, salvando sua chave substituta (unknown_loc_key) para uso posterior na carga da tabela fato.

```
[16]: with connect(connection_string) as conn:
      with conn.cursor() as cur:
          insert_query = sql.SQL("INSERT INTO dw.dim_loc (num_lat, num_lon,
          ↳nom_bai, grp_bai) VALUES (%s, %s, %s, %s)")
          cur.executemany(insert_query, [tuple(x) for x in df_loc.to_numpy()])
          cur.execute("INSERT INTO dw.dim_loc (num_lat, num_lon, nom_bai,
          ↳grp_bai) VALUES (NULL, NULL, NULL, NULL) RETURNING srk_loc")
          unknown_loc_key = cur.fetchone()[0]
```

Esta célula carrega os dados da dimensão Propriedade no schema dw. Ela insere os registros do DataFrame df_prop na tabela DIM_PROPRIEDADE e adiciona um registro com valores nulos para representar propriedades desconhecidas, salvando sua chave substituta (unknown_prop_key) que será usada na inserção da tabela fato.

```
[17]: with connect(connection_string) as conn:
      with conn.cursor() as cur:
          insert_query = sql.SQL("INSERT INTO dw.dim_pro (nom_anu, tip_qto,
          ↳qtd_min_noi, des_pol_can, ind_res_ins, ano_con, ind_tem_reg) VALUES (%s, %s,
          ↳%s, %s, %s, %s, %s)")
          cur.executemany(insert_query, [tuple(x) for x in df_prop.to_numpy()])
          cur.execute("INSERT INTO dw.dim_pro (nom_anu, tip_qto, qtd_min_noi,
          ↳des_pol_can, ind_res_ins, ano_con, ind_tem_reg) VALUES (NULL, NULL, NULL,
          ↳NULL, NULL, NULL, NULL) RETURNING srk_pro")
          unknown_prop_key = cur.fetchone()[0]
```

Esta célula realiza a carga da dimensão Última Avaliação no schema dw. Ela insere os registros do DataFrame df_aval na tabela DIM_ULTIMA_AVALIACAO e adiciona um registro com valores nulos para representar avaliações ausentes, armazenando a chave substituta (unknown_aval_key)

que será utilizada posteriormente na carga da tabela fato.

```
[18]: with connect(connection_string) as conn:
      with conn.cursor() as cur:
          insert_query = sql.SQL("INSERT INTO dw.dim_ult_ava (dat_ava, num_ano, num_mes, num_tri) VALUES (%s, %s, %s, %s)")
          cur.executemany(insert_query, [tuple(x) for x in df_aval.to_numpy()])
          cur.execute("INSERT INTO dw.dim_ult_ava (dat_ava, num_ano, num_mes, num_tri) VALUES (NULL, NULL, NULL, NULL) RETURNING srk_ava")
          unknown_aval_key = cur.fetchone()[0]
```

Esta célula faz o mapeamento das chaves substitutas (SRKs) das dimensões para o DataFrame principal. Ela lê as tabelas dimensionais do banco, realiza os joins com o DataFrame original (df) e substitui valores ausentes pelas chaves “desconhecidas”. O resultado é o DataFrame df_fato, já com todas as referências dimensionais resolvidas e pronto para ser inserido na tabela fato FATO_ANUNCIO.

```
[19]: with connect(connection_string) as conn:
      df_anfi_com_chaves = pd.read_sql("SELECT * FROM dw.dim_anf", conn)
      df_loc_com_chaves = pd.read_sql("SELECT * FROM dw.dim_loc", conn)
      df_prop_com_chaves = pd.read_sql("SELECT * FROM dw.dim_pro", conn)
      df_aval_com_chaves = pd.read_sql("SELECT * FROM dw.dim_ult_ava", conn)

      df_aval_com_chaves['dat_ava'] = pd.to_datetime(df_aval_com_chaves['dat_ava'])

      df_m = df.copy()
      df_m = pd.merge(df_m, df_anfi_com_chaves.drop_duplicates(subset=cols_anfi), on=cols_anfi, how='left')
      df_m = pd.merge(df_m, df_loc_com_chaves.drop_duplicates(subset=cols_loc), on=cols_loc, how='left')
      df_m = pd.merge(df_m, df_prop_com_chaves.drop_duplicates(subset=cols_prop), on=cols_prop, how='left')
      df_m = pd.merge(df_m, df_aval_com_chaves.drop_duplicates(subset=[col_aval]), on=col_aval, how='left')

      df_m['srk_anf'] = df_m['srk_anf'].fillna(unknown_anfi_key).astype(int)
      df_m['srk_loc'] = df_m['srk_loc'].fillna(unknown_loc_key).astype(int)
      df_m['srk_pro'] = df_m['srk_pro'].fillna(unknown_prop_key).astype(int)
      df_m['srk_ava'] = df_m['srk_ava'].fillna(unknown_aval_key).astype(int)

      cols_fato = ['qtd_dia_dis', 'val_pre', 'val_tax_ser', 'qtd_tot_ava', 'qtd_ava_mes', 'val_not_ava', 'srk_anf', 'srk_loc', 'srk_ava', 'srk_pro']
      df_fato = df_m[cols_fato].copy()
```

Esta célula realiza a carga final da tabela fato FATO_ANUNCIO no schema dw. Ela insere todos os registros do DataFrame df_fato, já com as chaves substitutas das dimensões, consolidando os dados no modelo estrela do Data Warehouse.

```
[20]: with connect(connection_string) as conn:
    with conn.cursor() as cur:
        insert_query = sql.SQL("""
            INSERT INTO dw.fat_anu (
                qtd_dia_dis, val_pre, val_tax_ser, qtd_tot_ava,
                qtd_ava_mes, val_not_ava,
                srk_anf, srk_loc, srk_ava, srk_pro
            ) VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s)
            """)

        cols = [
            'qtd_dia_dis', 'val_pre', 'val_tax_ser', 'qtd_tot_ava', 'qtd_ava_mes', 'val_not_ava',
            'srk_anf', 'srk_loc', 'srk_ava', 'srk_pro']

        def _to_db(v):
            if v is None:
                return None
            if isinstance(v, float) and math.isnan(v):
                return None
            if pd.isna(v):
                return None
            return v

        rows = [
            tuple(_to_db(v) for v in row)
            for row in df_fato[cols].itertuples(index=False, name=None)
        ]

        cur.executemany(insert_query, rows)
```

Esta célula consulta diretamente o banco de dados para verificar a quantidade de registros inseridos em cada tabela. Ela exibe o total de linhas carregadas nas dimensões e na tabela fato, funcionando como uma checagem final para confirmar que a etapa de carga foi concluída com sucesso.

```
[21]: with connect(connection_string) as conn:
    with conn.cursor() as cur:
        cur.execute("SELECT COUNT(*) FROM dw.dim_anf"); dim_anfi_count = cur.
        ↳fetchone()[0]
        cur.execute("SELECT COUNT(*) FROM dw.dim_loc"); dim_loc_count = cur.
        ↳fetchone()[0]
        cur.execute("SELECT COUNT(*) FROM dw.dim_pro"); dim_prop_count = cur.
        ↳fetchone()[0]
        cur.execute("SELECT COUNT(*) FROM dw.dim_ult_ava"); dim_aval_count =
        ↳cur.fetchone()[0]
        cur.execute("SELECT COUNT(*) FROM dw.fat_anu"); fato_count = cur.
        ↳fetchone()[0]
    print(f"Registros carregados no banco:")
```

```
print(f"dim_anf: {dim_anfi_count}")
print(f"dim_loc: {dim_loc_count}")
print(f"dim_pro: {dim_prop_count}")
print(f"dim_ult_ava: {dim_aval_count}")
print(f"fat_anu: {fato_count}")
```

Registros carregados no banco:

dim_anf: 25736

dim_loc: 65409

dim_pro: 93537

dim_ult_ava: 2437

fat_anu: 99417

[]: